

# AI Mini Project

## AI-based Sudoku Solver with Extended Constraints

---

**Department of Computer Science & Engineering**  
**National Institute of Technology, Patna**

**Submitted By:**

Name: \_\_\_\_\_ Roll No.: \_\_\_\_\_

Name: \_\_\_\_\_ Roll No.: \_\_\_\_\_

Name: \_\_\_\_\_ Roll No.: \_\_\_\_\_

Course Code : \_\_\_\_\_

Course Title: \_\_\_\_\_

Branch : Computer Science and Engineering

# Table of Contents

1. Aim of the Experiment
2. Introduction
  - 2.1 Background
  - 2.2 Problem Statement
  - 2.3 Objective
  - 2.4 Scope and Limitations
3. Theoretical Background
  - 3.1 Sudoku as a Constraint Satisfaction Problem
  - 3.2 Extended Constraints (Diagonal, Knight Moves)
  - 3.3 State-Space Search and Backtracking
  - 3.4 Constraint Propagation and Inference
4. Requirements
  - 4.1 Hardware Requirements
  - 4.2 Software Requirements
5. Design
  - 5.1 System Overview
  - 5.2 Data Structures
  - 5.3 Algorithm Flow
6. Implementation (Code)
7. Sample Input and Output
8. Observations
9. Conclusion
10. Learning Outcomes
11. References

# 1. Aim of the Experiment

The aim of this mini project is to design and implement an AI-based Sudoku solver for an extended version of the puzzle that supports additional logical constraints beyond the standard row, column, and subgrid rules. The solver uses state-space search, constraint propagation, inference, and backtracking to find a valid completed grid.

## 2. Introduction

### 2.1 Background

Sudoku is a popular logic-based puzzle played on a  $9 \times 9$  grid partitioned into  $3 \times 3$  subgrids. The goal is to fill the grid so that each row, each column, and each subgrid contains all digits from 1 to 9 exactly once. Because of its clear rules and combinatorial structure, Sudoku is a classic example of a constraint satisfaction problem and a benchmark for AI search techniques.

### 2.2 Problem Statement

Design and implement a Python program that can solve an extended Sudoku puzzle where, in addition to the standard row, column, and subgrid constraints, extra rules such as diagonal constraints and knight-move restrictions may also be imposed. The solver should accept a partially filled grid as input and produce a valid completed solution satisfying all active constraints.

### 2.3 Objective

The objectives of this project are:

- To model Sudoku as a state-space search and constraint satisfaction problem.
- To incorporate extended constraints such as diagonal and knight-move rules.
- To implement a backtracking search with constraint propagation and inference in Python.
- To demonstrate how richer rule sets can still be handled systematically using AI techniques.

### 2.4 Scope and Limitations

The implementation focuses on  $9 \times 9$  Sudoku puzzles. The solver supports standard Sudoku constraints and can optionally enable main-diagonal, anti-diagonal, and knight-move restrictions. Other exotic constraints are not implemented, but the design is modular so that additional rules could be added. Performance is suitable for typical puzzles but not optimized for the hardest known instances.

## **3. Theoretical Background**

### **3.1 Sudoku as a Constraint Satisfaction Problem**

A constraint satisfaction problem (CSP) consists of a set of variables, domains of possible values for each variable, and a set of constraints specifying which combinations of values are allowed. In Sudoku, each cell of the grid is a variable, the domain is the set of digits 1–9 for empty cells, and the constraints enforce that no digit repeats in any row, column, or subgrid.

### **3.2 Extended Constraints (Diagonal, Knight Moves)**

Extended Sudoku variants introduce extra logical rules to increase difficulty and variety. Diagonal Sudoku requires that the two main diagonals also contain digits 1–9 without repetition. Knight-move Sudoku adds a chess-like restriction: any two cells separated by a knight's move (two steps in one direction and one perpendicular) cannot contain the same digit. These constraints tie distant cells together, creating a richer dependency structure.

### **3.3 State-Space Search and Backtracking**

The space of all possible grid configurations can be viewed as a state space, where each state represents a partially filled board. Backtracking search incrementally assigns values to empty cells, moving forward when constraints are satisfied and backtracking when a conflict is encountered. This depth-first exploration continues until a complete and consistent assignment is found or all possibilities are exhausted.

### **3.4 Constraint Propagation and Inference**

Constraint propagation reduces the search space by eliminating values from variable domains that cannot appear due to current assignments. In Sudoku, once a digit is placed in a cell, it is removed from the possible candidates of all related cells in the same row, column, subgrid, diagonals, or knight-move positions. Simple inference rules, such as choosing the cell with the fewest remaining candidates next, can greatly speed up the search.

## 4. Requirements

### 4.1 Hardware Requirements

- A computer system with at least:
  - Processor: Intel i3 or equivalent
  - RAM: 4 GB or more
  - Storage: 500 MB free disk space
- Any modern operating system capable of running Python.

### 4.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Python Interpreter: Python 3.x
- Text editor or IDE for writing and running Python code (e.g., VS Code, PyCharm, IDLE).

## 5. Design

### 5.1 System Overview

The solver is implemented as a Python module. A  $9 \times 9$  grid is represented as a list of lists, where 0 denotes an empty cell. The solver reads an initial grid, then repeatedly selects an empty cell, assigns a candidate digit consistent with all active constraints, and continues recursively. If no candidate works, the algorithm backtracks to a previous cell and tries a different value.

### 5.2 Data Structures

- Board: a  $9 \times 9$  list of integers.
- Constraint flags: boolean options indicating whether diagonal and knight-move rules are enabled.
- Helper functions: routines to find empty cells, test consistency of a candidate digit, and print the board.

### 5.3 Algorithm Flow

High-level flow:

Start  $\rightarrow$  Read/define initial grid  $\rightarrow$  Call solve()  $\rightarrow$  If all cells assigned consistently, print solution; otherwise report that no solution exists. During solve(), recursively place digits with backtracking and constraint checks at each step.

## 6. Implementation (Code)

Below is the Python implementation of the extended-constraint Sudoku solver.

```
# AI Mini Project: Extended-Constraint Sudoku Solver

from copy import deepcopy

class SudokuSolver:
    def __init__(self, board, diagonal=False, knight_moves=False):
        self.board = deepcopy(board)
        self.size = 9
        self.diagonal = diagonal
        self.knight_moves = knight_moves

    def find_empty(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c
        return None

    def is_valid(self, row, col, val):
        # Row and column
        for i in range(self.size):
            if self.board[row][i] == val:
                return False
            if self.board[i][col] == val:
                return False

        # Subgrid
        box_r = (row // 3) * 3
        box_c = (col // 3) * 3
        for r in range(box_r, box_r + 3):
            for c in range(box_c, box_c + 3):
                if self.board[r][c] == val:
                    return False

        # Diagonal constraints (if enabled)
        if self.diagonal:
            if row == col:
                for i in range(self.size):
                    if self.board[i][i] == val:
                        return False
            if row + col == self.size - 1:
                for i in range(self.size):
                    if self.board[i][self.size - 1 - i] == val:
                        return False

        # Knight-move constraints (if enabled)
        if self.knight_moves:
            knight_offsets = [
                (-2, -1), (-2, 1),
                (-1, -2), (-1, 2),
                (1, -2), (1, 2),
                (2, -1), (2, 1),
            ]
            for dr, dc in knight_offsets:
                nr, nc = row + dr, col + dc
                if 0 <= nr < self.size and 0 <= nc < self.size:
                    if self.board[nr][nc] == val:
                        return False

        return True

    def solve(self):
```

```

        empty = self.find_empty()
        if not empty:
            return True # solved

        row, col = empty

        # Simple heuristic: try values 1..9
        for val in range(1, 10):
            if self.is_valid(row, col, val):
                self.board[row][col] = val
                if self.solve():
                    return True
                self.board[row][col] = 0

        return False

def print_board(self):
    for r in range(self.size):
        if r % 3 == 0 and r != 0:
            print("-----+-----+-----")
        row_vals = []
        for c in range(self.size):
            if c % 3 == 0 and c != 0:
                row_vals.append("|")
            val = self.board[r][c]
            row_vals.append(str(val) if val != 0 else ".")
        print(" ".join(row_vals))

if __name__ == "__main__":
    # 0 represents an empty cell
    puzzle = [
        [0, 0, 0, 2, 6, 0, 7, 0, 1],
        [6, 8, 0, 0, 7, 0, 0, 9, 0],
        [1, 9, 0, 0, 0, 4, 5, 0, 0],
        [8, 2, 0, 1, 0, 0, 0, 4, 0],
        [0, 0, 4, 6, 0, 2, 9, 0, 0],
        [0, 5, 0, 0, 0, 3, 0, 2, 8],
        [0, 0, 9, 3, 0, 0, 0, 7, 4],
        [0, 4, 0, 0, 5, 0, 0, 3, 6],
        [7, 0, 3, 0, 1, 8, 0, 0, 0],
    ]

    # Enable diagonal or knight-move constraints as needed
    solver = SudokuSolver(puzzle, diagonal=True, knight_moves=False)

    print("Initial puzzle:")
    solver.print_board()

    if solver.solve():
        print("\nSolved puzzle:")
        solver.print_board()
    else:
        print("No solution exists for the given constraints.")

```

## 7. Sample Input and Output

A sample console run of the program is illustrated below (truncated for brevity).

```
Initial puzzle:  
. . . 2 6 . 7 . 1  
6 8 . . 7 . . 9 .  
1 9 . . . 4 5 . .  
...
```

```
Solved puzzle:  
4 3 5 2 6 9 7 8 1  
6 8 2 5 7 1 4 9 3  
1 9 7 8 3 4 5 6 2  
...
```

## 8. Observations

- The solver quickly completes typical Sudoku puzzles using backtracking and constraint checks.
- Enabling diagonal or knight-move constraints increases the difficulty but the same framework still works.
- Choosing a reasonable order of cell assignments has a noticeable impact on performance.
- The modular constraint checks make it easy to experiment with additional puzzle rules.

## 9. Conclusion

This mini project demonstrates how AI techniques such as state-space search, constraint satisfaction, and backtracking can be applied to solve structured reasoning problems like extended Sudoku. By treating each cell as a variable with a constrained domain and applying systematic search with inference, the solver is able to handle richer rule sets than traditional Sudoku while maintaining a clear and extensible design.

## 10. Learning Outcomes

Through this experiment, the following concepts and skills were learned:

- Modeling Sudoku as a constraint satisfaction problem.
- Implementing state-space search and backtracking in Python.
- Incorporating additional logical constraints such as diagonals and knight moves.
- Understanding the role of constraint propagation and inference in reducing the search space.

## 11. References

1. Stuart Russell and Peter Norvig, "Artificial Intelligence: A Modern Approach", Pearson.
2. Standard online resources on Sudoku, CSPs, and backtracking algorithms.
3. Documentation of the Python programming language (<https://docs.python.org/3/>).